Joseph Dye, Simon Glashauser, Nick Russell, Mason Thomas
CSC 2400-001
C. Radian
27 April 2026

The Exam Scheduling Problem: A Comparative Analysis of Algorithms

The Exam Scheduling Problem is an optimization problem that aims to create a conflict free exam timetable where no student is scheduled to be in multiple exams at the same time and no exams occupy the same location at the same time. The problem in this instance will consider the following constraints: exam length, student schedules, location, and time for when exams can be held. The idea is that, given a set of exams for classes at a university, no student is scheduled to be in two exams at once, no exams happen in the same space at the same time, and all exams occur within a certain time of the day over a certain number of days.

While many solutions to the Exam Scheduling Problem may exist, this report examines four algorithms: exhaustive brute force, a greedy algorithm, a graph coloring algorithm, and a genetic algorithm. A brief description of these four algorithms is as follows:

The brute force approach generates all permutations of schedules given the number of exams, students, and locations. For every schedule created, it immediately checks if that schedule is valid, returning that scheduling and ending if so and continuing with permutations if not.

The greedy algorithm creates a roster for each course where each course includes which students enrolled. Starting with the exam (course) with the most students, it selected the next available timeslot with the fewest number of conflicts. While this algorithm is fast, because it makes an assignment every pass it can be inaccurate.

The graph coloring algorithm creates a student-exam roster akin to that of the greedy algorithm. A conflict graph is built where each vertex is an exam, and each edge is a shared student between two exams. The idea is that if two exams have an edge, they have a conflict and thus should not be scheduled at the same time. The algorithm then follows a degree of saturation (DSatur) technique that schedules the exam with the least number of conflicts (edges) first before continuing to the next vertex.

The genetic algorithm represents data via chromosomes (each a full schedule) and genes (each a timeslot). A fitness value is assigned to the chromosome based on the number of conflicts, meaning every time a student or exam is improperly scheduled, the fitness score is decreased. The algorithm then takes a pool from the population of schedules, keeps the best, crosses it over with another schedule from another pool, and introduces random mutation to adjust some exam timeslots (genes). This process is repeated until the generation count is exceeded or a perfect fitness score, indicating no conflicts, is found.

The objective of this report revolved around seeing which of these algorithms works the best for generating schedules, not only creating schedules with minimal conflicts but also in a reasonable runtime.

The following research questions could be used to analyze these aspects:

1. How does adjusting the population size and generation count affect the quality of the output of the genetic algorithm, and is it worth computational costs?

   It is expected that this will increase the quality of the output but will greatly increase runtime. It may be beneficial at small input sizes but could grow very fast at large input sizes.

2. In what cases would we expect the greedy algorithm to outperform the graph coloring algorithm?

   It is expected that the greedy will outperform the graph coloring when preprocessing is not an advantage, such as if a schedule has lots of valid solutions. This way, any decision made with the greedy algorithm will likely not create conflicts

3. How does decreasing the number of colors (timeslots) in a graph coloring problem impact the runtime and memory requirements of the algorithm?

   It is expected that decreasing the timeslots will decrease the runtime of the algorithm because there are less connections to check. It will likely also produce more conflicts.

To implement our solution, we used a mixture of Python and C code. We utilized Python for user-facing applications such as the main menu and data generation. Python was chosen for this task because of its quick adaptability, ease of implementation, and the lack of a need for highly performant code in this use case. We utilized C to implement all algorithms solely because of its high performance. Brute force and the genetic algorithm in particular have at least many millions of basic operations, which means that even a small increase in efficiency means a huge decrease in overall runtime. In the order of brute force, greedy, graph coloring, and genetic, their basic operations were: a singular schedule generated, a course placed in a slot, an edge colored, and a singular schedule generated per individual.

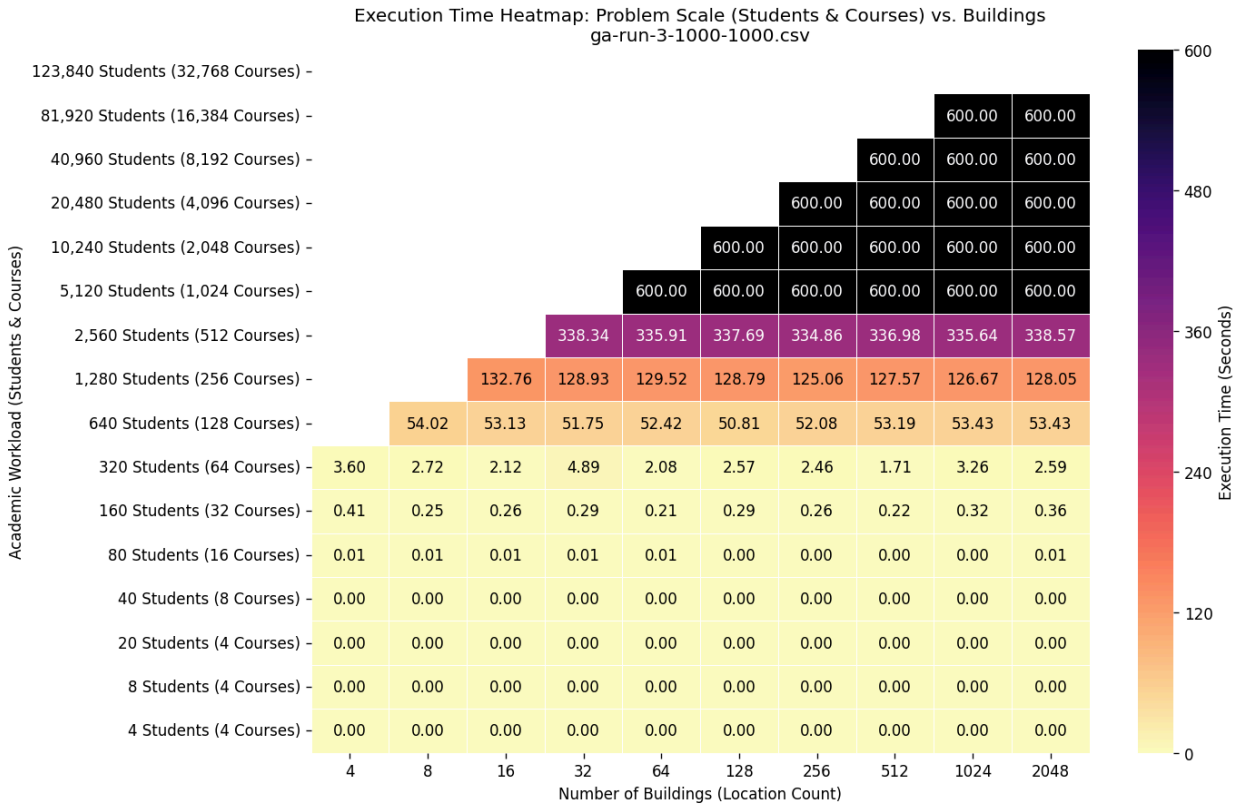
Our inputs were all artificial datasets generated randomly by our own data creation Python scripts. We chose to use pure artificial datasets because of the lack of readily available in student enrollment datasets. For the datasets we were able to find, we found that simply normalizing the data would have been a project within itself. These datasets created courses at random. Student datasets could then be created off of a course dataset, with each student being assigned a department as "major" by a weighted system to reflect real-world differences in department sizes. Based on their major, for each course, students were 80% likely to choose a course from their department. Otherwise, they would randomly pick a course from a different department determined by the same weighted system as before. This system was implemented to

reflect when students must take general education or other elective courses outside of their major.

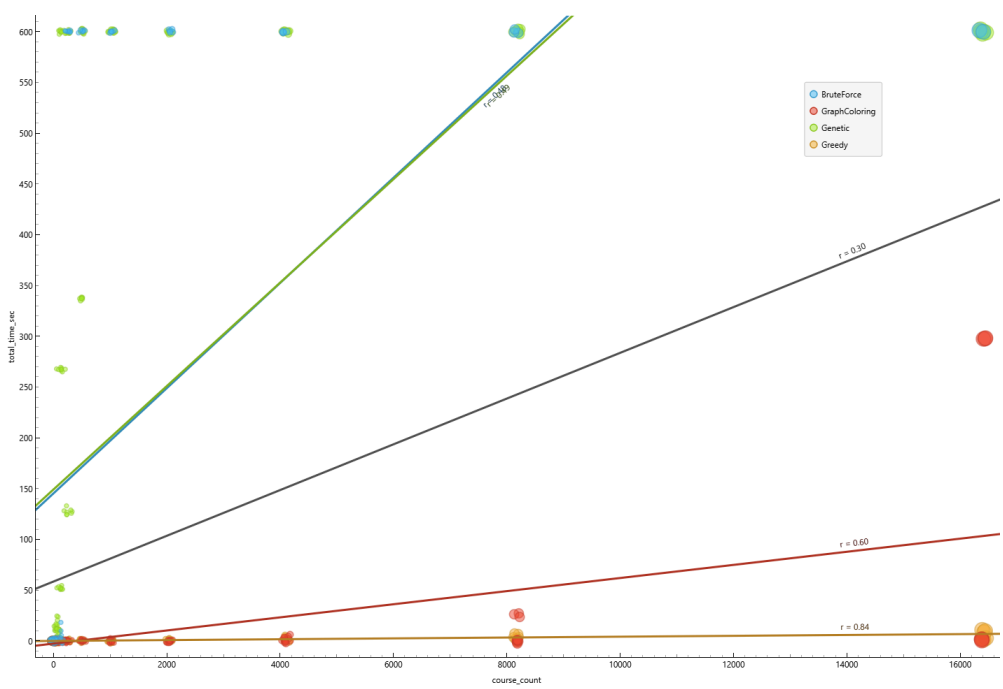
The algorithms were all run with the same 115 combinations of student counts, course counts, and locations. Greedy and graph coloring were run 3 times in a row while brute force and genetic were only run once per variation. This was solely due to the former's runtime being quite low so extra data was generated to account for one-time flukes in the data. The latter took too long to run for us to provide them with the same affordance. This was managed by a 'benchmark.py' script that called each algorithm, captured its results, and input that into a CSV file. This script also managed the timeout for each algorithm, which was also saved as a datapoint if achieved. The timeout for a single run of each algorithm was set for 10 minutes. While this value is significantly lower than we would've liked, it was chosen due to its ability to have all algorithms fully complete all 115 combinations in ~8 hours. This is significant as that is enough time to be able to run an algorithm overnight so the computer may be used for other purposes the next day. All data analyzed for this project was generated by running the algorithms on an M2 MacBook Pro. This model has 12-CPU cores, 4 efficiency focus cores, and 8 performance focus cores and 19 general-use GPU cores. The user is unable to direct which core algorithms run on so it is unknown if efficiency or performance cores were utilized more. This model also has 16GB of RAM with ~100GB of SSD swap available, and it never ran out of memory.

When evaluating our algorithms, we primarily looked at the number of student conflicts and the runtime of the algorithm. The lower both of these values, the closer the algorithm was to the most optimal solution. Since many solutions can be equally as valid, comparing all algorithms to one another was the most effective method of comparison. We used brute force as the baseline for that a given solution for the same input size should never be worse than it. Broadly, the most optimal solution to this problem is not widely agreed upon. For our purposes of comparison, as mentioned earlier, we treated the most optimal solution as the one with the lowest number of student conflicts and runtime.

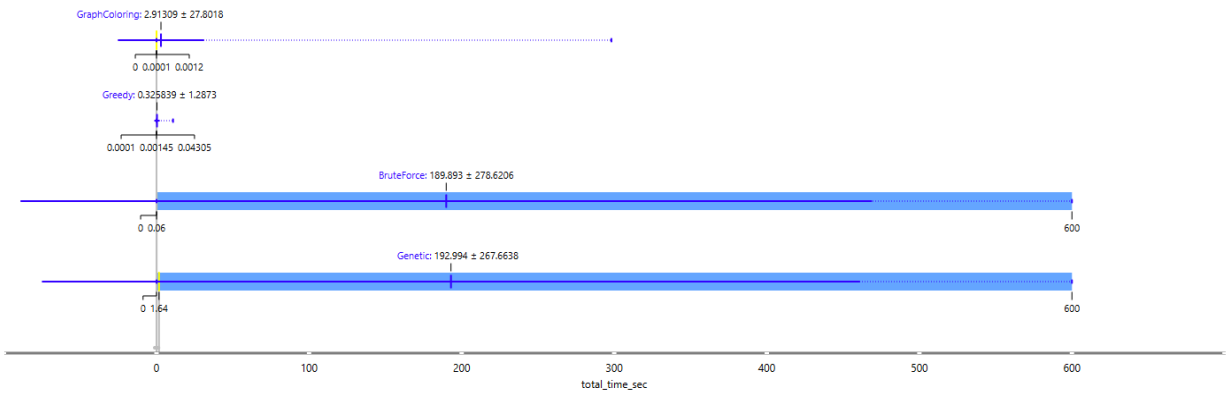
The data collected described above is best represented through heatmaps. The heatmap shown below is one iteration of datasets run (Heatmap 10: the genetic algorithm with 1000 generations and a population size of 1000). It shows the relationship between the number of locations, number of students/courses, and the runtime of the algorithm. Since running the data had a timeout of 600 seconds, anything labeled as such timed out. This heatmap clearly shows the effect increasing input size has on the runtime of the algorithm, which is fully explained later in the report.



The scatterplot to the right (Scatterplot 1) shows all the data collected, thus showing relations between algorithms. Comparisons with how each algorithm runs are shown, with the greedy and graph coloring being closely related and the brute force and genetic also being closely related. The x-axis is a measure of the number of exams (primary independent variable) and the y-axis is a measure of runtime. The size of the data points is correlated to the number of locations in that dataset, and the colors represent which algorithm was run (see key). Slight jittering was applied to distinguish results with similar positions.



The boxplot shown below (Boxplot 1) compares the runtime of the four algorithms across all recorded data. Using the median of the datasets, it is clear that graph coloring and greedy had superior and near-constant runtimes while brute force and the genetic algorithm has significantly more variance.



More visualizations can be seen in the Appendix of this report. The above charts and graphs are shown again alongside heatmaps for all datasets generated.

Regarding the results of the four algorithms, the empirical analysis in terms of runtime, student conflicts, and course conflicts provided the expected results for our Greedy algorithm, but was slightly different for the coloring algorithm, and timed out the Brute Force and Genetic algorithms.

The Brute Force algorithm always got the correct results but started timing out at 640 students and 128 exams due to its exhaustive approach. For a small input size its average runtime was good, but as its size increased it became exponentially slower because it evaluates every possible schedule permutation before settling on a solution.

The greedy algorithm ran relatively quickly with an average runtime of 0.338803 seconds. It provided a perfect set of data for every student and course count with a valid number of locations, with the only conflicts arising at student size of 81920 and a course list size of 16384.

The Graph Coloring algorithm ran in a much faster time with an average of 0.01846422 seconds. but didn’t match the greedy algorithm's results. It correctly assigned exams to rooms, but student conflicts arose due to the lack of rooms to accommodate for each student's set of four randomly assigned exams during the post-processing algorithm.

Similarly to the Brute Force algorithm, the Genetic algorithm started with a great runtime for small sets of data but exponentially slowed down. It started providing incomplete data when it reached its upper limit of 1000000 generated schedules, and it started to time out with a population of 5120 students and 1024 exams.

As stated previously, we asked guiding research questions that analyze the performance of the exam scheduling algorithms. Empirically analyzing the data produced the following answers to the research questions:

1. How does adjusting the population size and generation count affect the quality of the output of the genetic algorithm, and is it worth computational costs?

    Unexpectedly, the answer to this question depends on whether you increase the population size, the generation count, or both. Each variation provided different output compared to one another. For this question, we will be comparing the run of 640 students, 128 courses, and varying numbers of locations. These parameters were chosen as this is where each variation begins to fail to produce a perfect schedule and some even timeout. To dive right in, increasing the population size alone actually decreased (~2%) the fitness of our final output while still increasing runtime by 500% when compared to our baseline. Increasing the generation count increased (~9.1%) the fitness score of our final output and increased runtime again by 500%. Finally, if you increase both, the algorithm begins to timeout before it would provide a final answer.

    Clearly, it is not advantageous or worth the added computational cost to increase the population size alone or increase both. We also argue that it is not worth the added computational cost to increase the generation count either. A 9% increase to fitness for the trade-off of a 500% increase to runtime is very poor. Other algorithms such as greedy, on average, ran ~200% quicker even when compared to the quickest run of any variation of the genetic at this input size. Additionally, in this case, greedy produced a conflict-free schedule while no variation of the genetic also at this input size. Overall, increasing the parameters of the genetic algorithm is clearly a careful art that does not offset computational costs even if you see an increase in fitness values.

2. In what cases would we expect the greedy algorithm to outperform the graph coloring algorithm?

    The answer to this question is also not what we expected. At small input sizes, when a large percentage of possible schedules are valid, we expected that greedy would outperform graph coloring; however, the opposite happened. A clear divide can be seen in the data where for each input size, when sorted by runtime, nearly all graph coloring runs are at the top while greedy runs are at the bottom. As we move up in input size, we see a clear pattern where graph coloring outperforms greedy except for when the number of locations available is within ~20% of the mathematical minimum number of locations needed for a given input size. Graph coloring run time begins to explode in this case.

    For example, when given 81,920 students and 16,384 courses, graph coloring produced a perfect schedule in only 1 second when given 2,048 locations. However, if you halve the number of locations to 1,024, the runtime

explodes 30,000% to nearly 300 seconds while still containing 18 student conflicts. Here, the minimum number of locations needed is 820. Decreasing 1024 by 20% gets you the exact same value. You can also recognize this pattern on a smaller scale for the input size 20,480 students and 4,096 students where the minimum location count is 205. When graph coloring is given only 256 locations, 20% away from the minimum, the runtime compared to when the location count is doubled, increased by ~8,000%. In conclusion, graph coloring will generally outperform the greedy. However, if the number of locations available is within ~20% of the absolute minimum of the number of locations needed, greedy will outperform graph coloring.

3. How does decreasing the number of colors (timeslots) in a graph coloring problem impact the runtime and memory requirements of the algorithm?

    Decreasing the number of colors (time slots) for the graph coloring algorithm unexpectedly increased the runtime of the algorithm. There was no meaningful difference in the memory requirement. To dive further in, when looking at the raw data, the average runtime for the "normal" graph coloring, with 20 time slots, was ~2.9 seconds and the "reduced", with 12 time slots, was ~0.11 seconds. However, you cannot directly compare these values. As spoken about in the previous research question, the runtime of graph coloring explodes when the number of time slots given is within 20% of the minimum number of time slots needed. With how our testing data was constructed, this forced the normal algorithm to operate within that zone. However, the reduced version lost 40% of its available time slots, leading it to be impossible for it to ever provide any solution for the exact same problem that the normal received.

    To solve this problem, we simply exclude those outliers from our comparison data set. Once outliers have been removed, the average runtime of the normal graph coloring is much lower at ~0.018 seconds. This is a ~500% difference in runtime when compared to the reduced version. It is also important to mention that the reduced version produced a substantially greater number of student conflicts than the normal version. The reduced version on average produced ~236 student conflicts while the normal version only produced ~0.38 student conflicts on average. Overall, reducing the number of time slots that graph coloring has massive effects on the runtime and quality of the output of the algorithm. It does not have any noticeable difference in its memory requirement.

Key limitations of our experiment include a lack of constraints on the number of students per room, a fixed number of courses each student takes, and the available amount of time and the resources to run the Brute Force and Genetic algorithms. Adding room capacity limits would likely increase the number of rooms required. A more realistic model for each student would assign between 4 and 7 courses weighted at 1–4 credit hours, better simulating a standard 12–21

credit hour load. Finally, access to more time or a supercomputer would allow a more full evaluation of the Brute Force and Genetic algorithms at larger scales.

If we were to do this all over again, we would replace exam assignments with realistic enrollment data and test a wider range of room-to-exam ratios since our empirical analysis showed low room availability significantly impacted the Graph Coloring algorithm. We would also fix its post-processing step to guarantee that it prevents student conflicts, and better tune the Genetic algorithm's parameters while running it on more powerful hardware. Finally, we would investigate how varied student course distributions affect scheduling outcomes.

These results also prompt several further research questions: How does enforcing room capacity as a hard constraint affect each algorithm's conflict rate and scalability? Would fixing the Graph Coloring algorithm's post-processing step to guarantee no student conflicts allow it to retain its speed advantage over the Greedy algorithm? And can the Genetic algorithm's parameters be tuned to produce better solutions than Greedy at medium-to-large input sizes?

Every member of our team made meaningful contributions to the project. This report and the repository represents the combined brain power and effort of all members, each coming to the table with a different skillset. How responsibilities were divided up among the team can be seen in the table below.

Task	**Joseph**	**Simon**	**Mason**	**Nick**
Brainstorming	35%	20%	35%	10%
Visualization/Results	10%	0%	40%	50%
Report Writing	40%	10%	40%	10%
Submission	50%	0%	50%	0%
Communication	25%	25%	25%	25%
Coding	30%	20%	30%	20%
Experiment Design	30%	30%	30%	10%
Presentation	60%	10%	10%	20%

Our main difficulties in this project came from computational limitations and modeling constraints. The Brute Force and Genetic algorithms timed out at larger input sizes due to their exhaustive and population-based approaches, preventing full evaluation across all scales. The Graph Coloring algorithm's post-processing step introduced unexpected student conflicts caused by insufficient room availability even though it ran the fastest. Additionally, the lack of access to more powerful hardware restricted how quickly and thoroughly we could test the more expensive algorithms.

Works Cited

Aspnes, James. "BruteForce." *PineWiki*, 17 June 2014, www.cs.yale.edu/homes/aspnes/pinewiki/BruteForce.html.

"Brute Force Approach and Its Pros and Cons." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/dsa/brute-force-approach-and-its-pros-and-cons/.

Elen, Abdullah. "A COMPLETE SOLUTION to EXAM SCHEDULING PROBLEM: A CASE STUDY." *Kutahya Dumlupmar University Institute of Graduate Studies*, Journal of Scientific Reports - A, 14 Oct. 2021, dergipark.org.tr/en/download/article-file/2025907.

"Genetic Algorithms." *GeeksforGeeks*, GeeksforGeeks, 10 Feb. 2026, www.geeksforgeeks.org/dsa/genetic-algorithms/.

"Graph Coloring Using Greedy Algorithm." *GeeksforGeeks*, GeeksforGeeks, 23 Mar. 2025, www.geeksforgeeks.org/dsa/graph-coloring-set-2-greedy-algorithm/.

"Greedy Algorithms Tutorial." *GeeksforGeeks*, GeeksforGeeks, 22 Jan. 2026, www.geeksforgeeks.org/dsa/greedy-algorithms/.

"Introduction to Graph Coloring." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/dsa/graph-coloring-applications/.

Lemos, Alexandre Rosseto Rosseto. "Genetic Algorithm and Its Practicality in Machine Learning." *Towards Data Science*, 5 Mar. 2025, towardsdatascience.com/genetic-algorithm-6aefd897f1ac/.

W3Schools. "DSA Greedy Algorithms." *W3Schools Online Web Tutorials*, www.w3schools.com/dsa/dsa_ref_greedy.php.

Artificial Intelligence Use

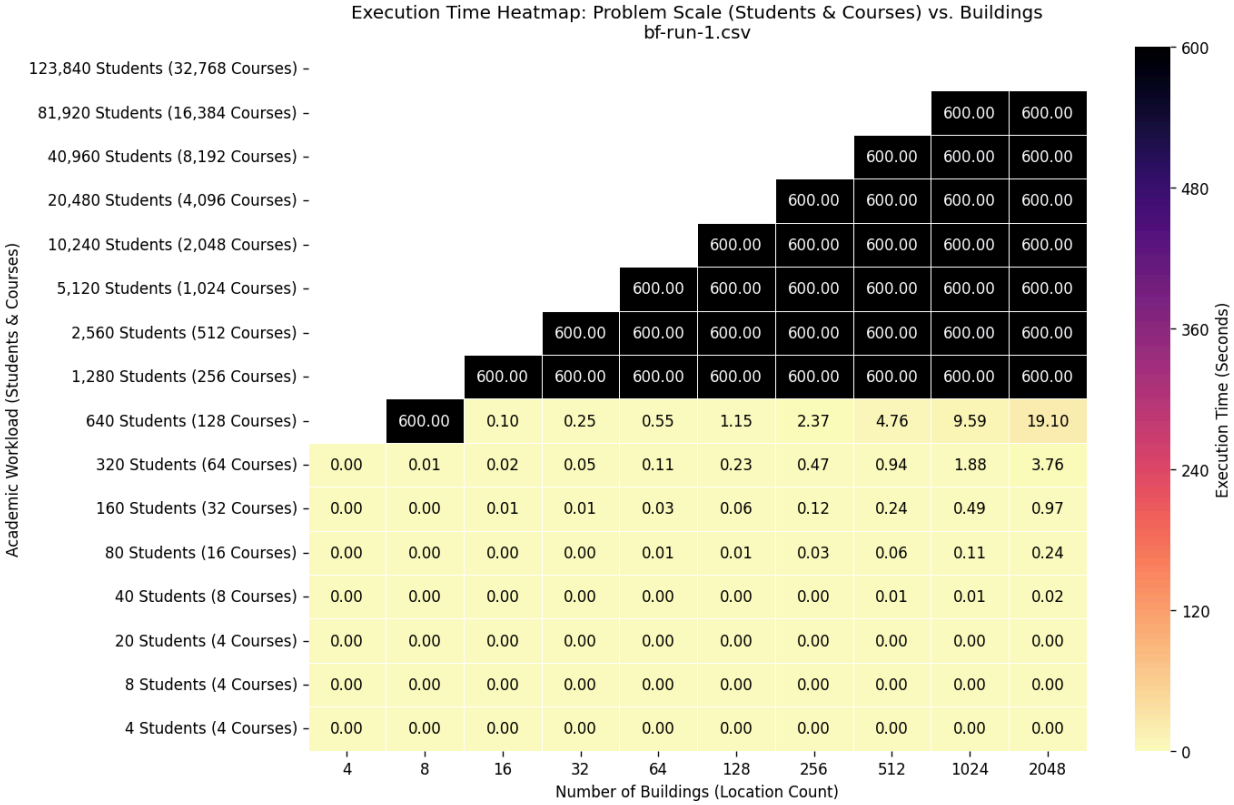
Claude: Used for generating C-based algorithms and the conversion of python algorithms into C

ChatGPT: Used for understanding the problem and editing of the used algorithms

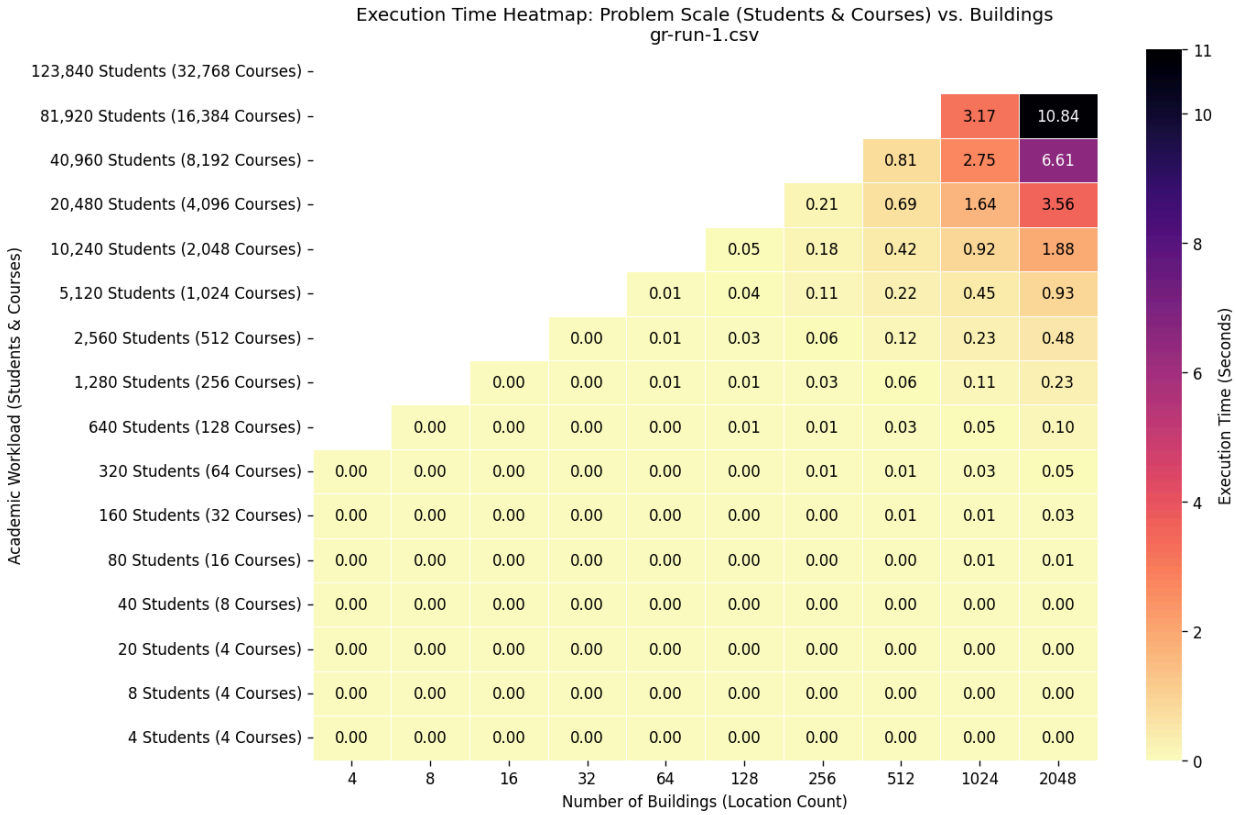
GitHub Copilot: Used for debugging and editing file paths in the algorithms

Appendix

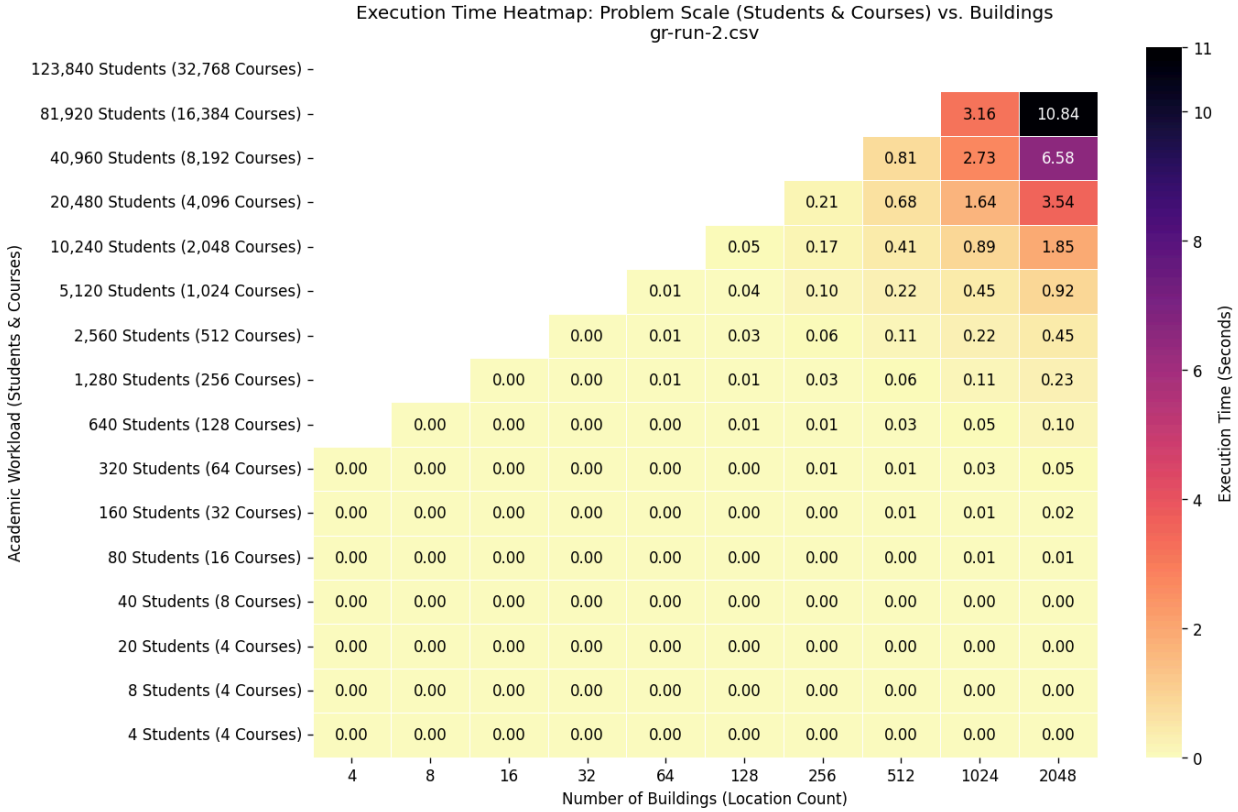
Heatmap 1: bf-run-1_heatmap (Brute Force run #1)



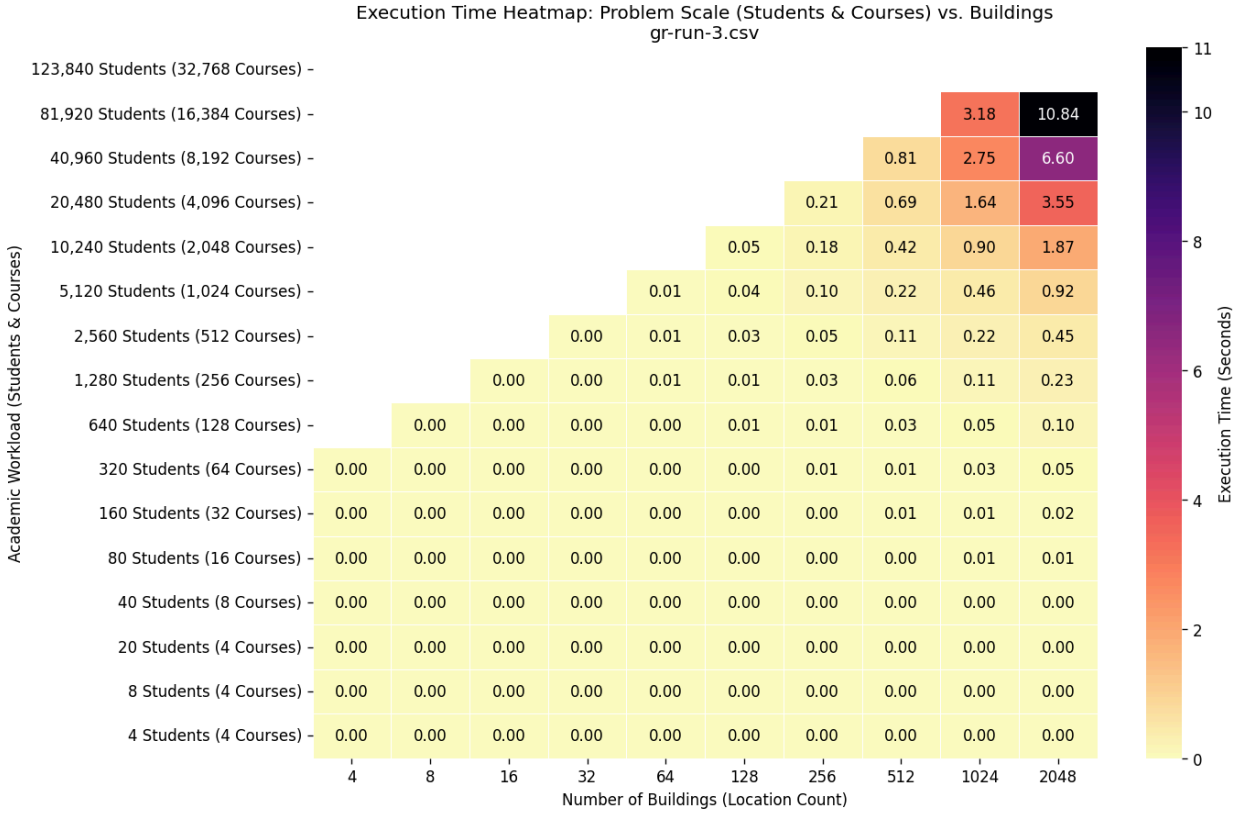
Heatmap 2: gr-run-1_heatmap (Greedy run #1)



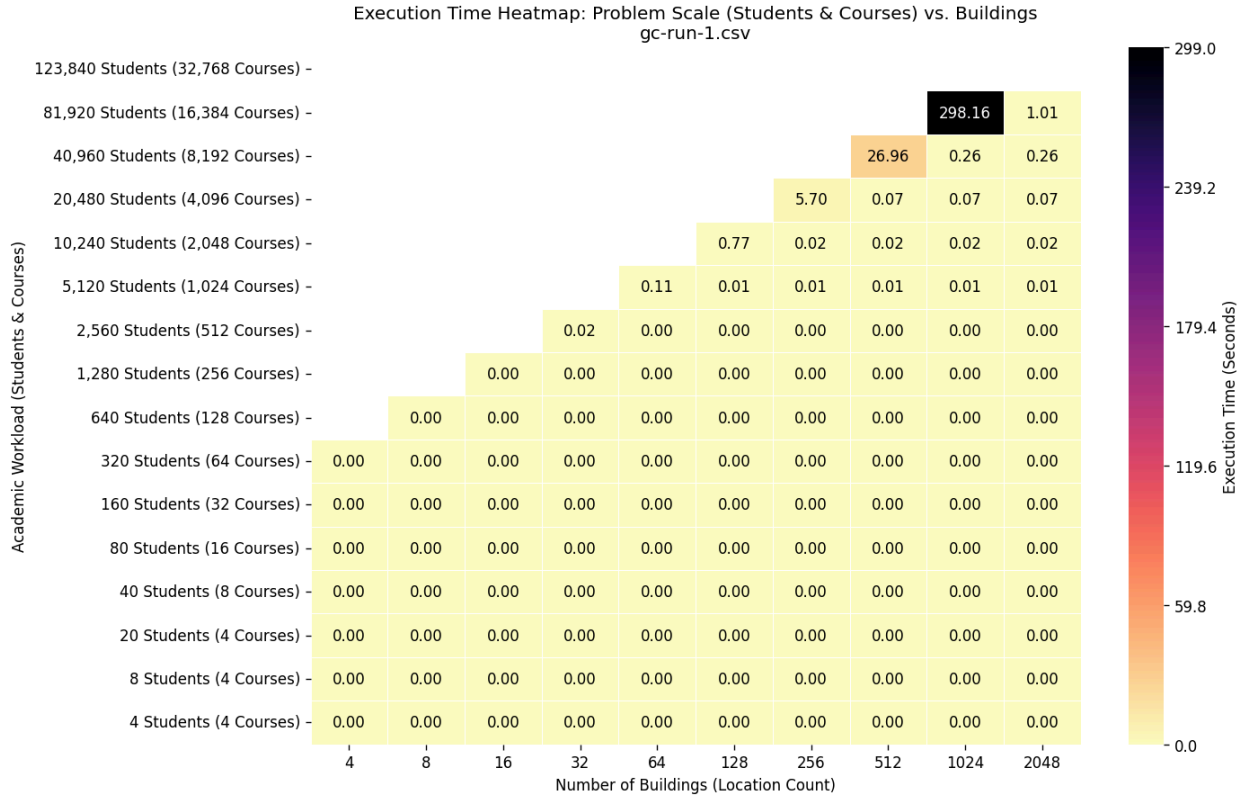
Heatmap 3: gr-run-2_heatmap (Greedy run #2)



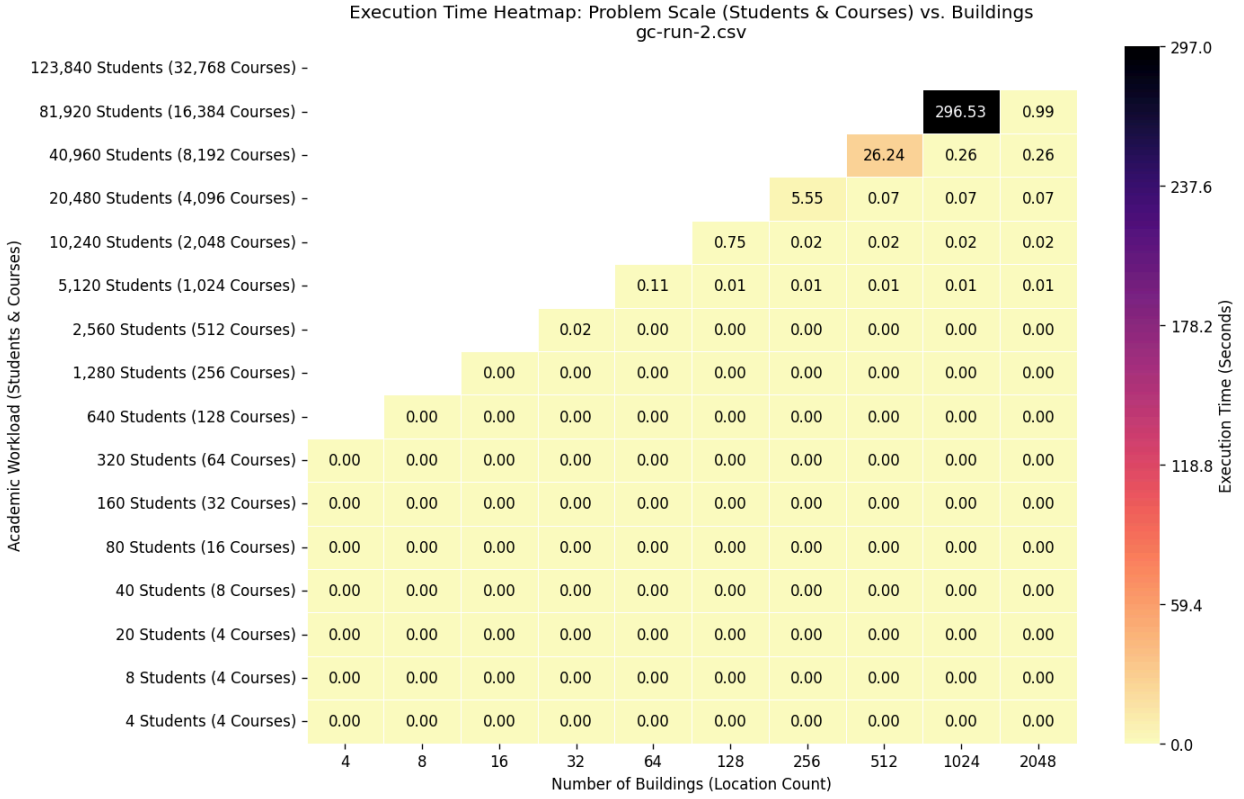
Heatmap 4: gr-run-3_heatmap (Greedy run #3)



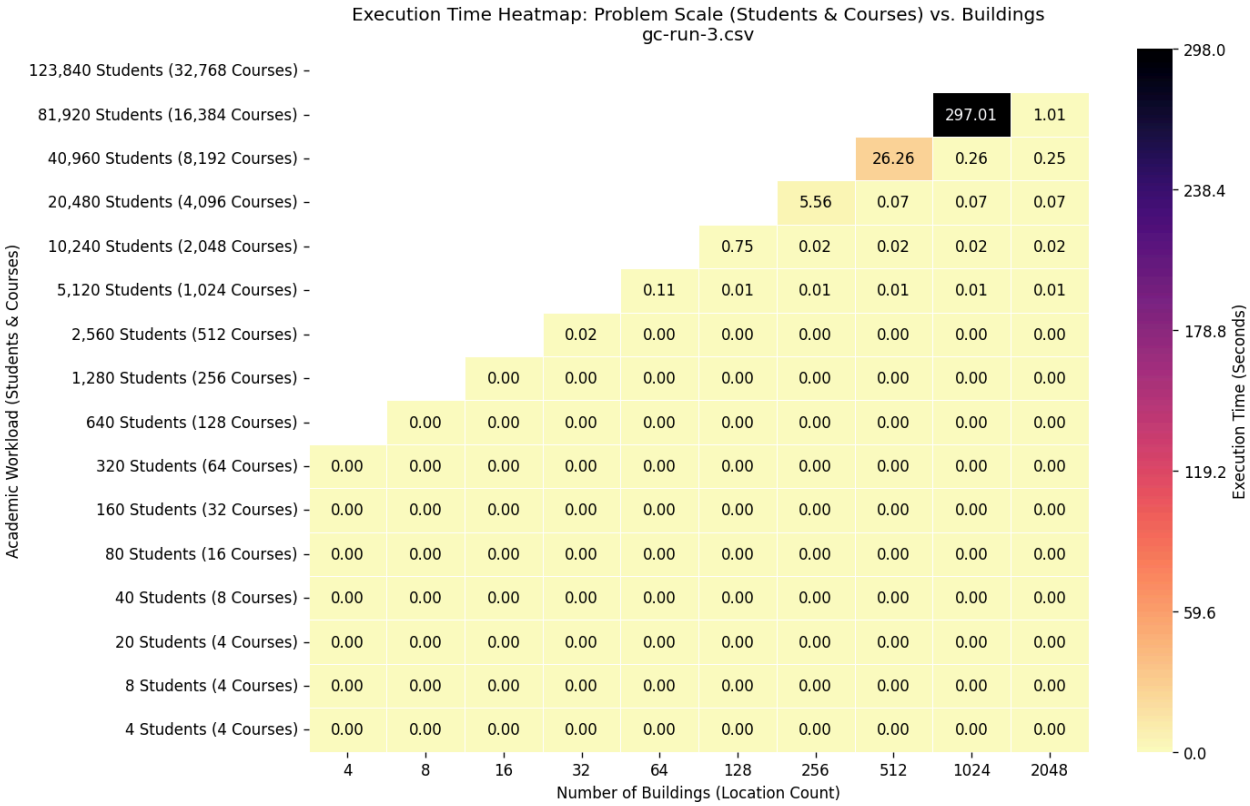
Heatmap 5: gc-run-1-heatmap (Graph Coloring run #1)



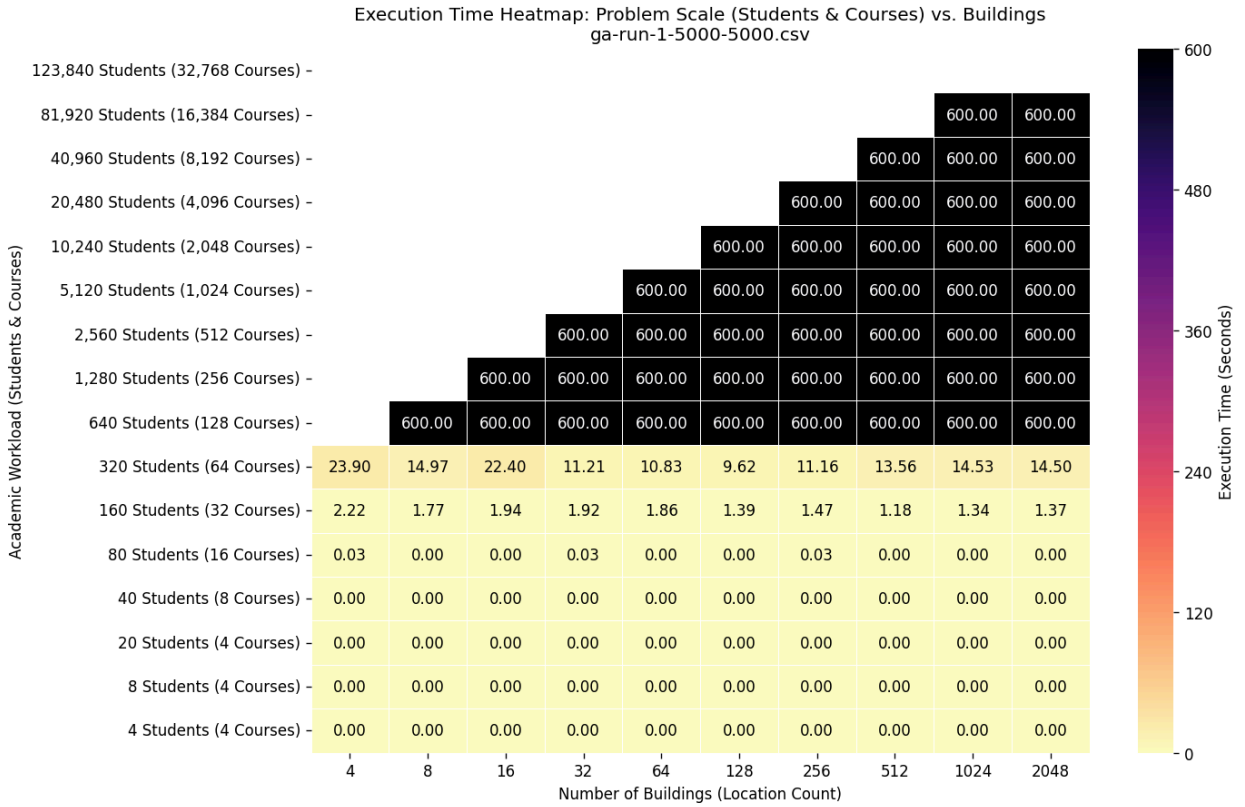
Heatmap 6: gc-run-2-heatmap (Graph Coloring run #2)



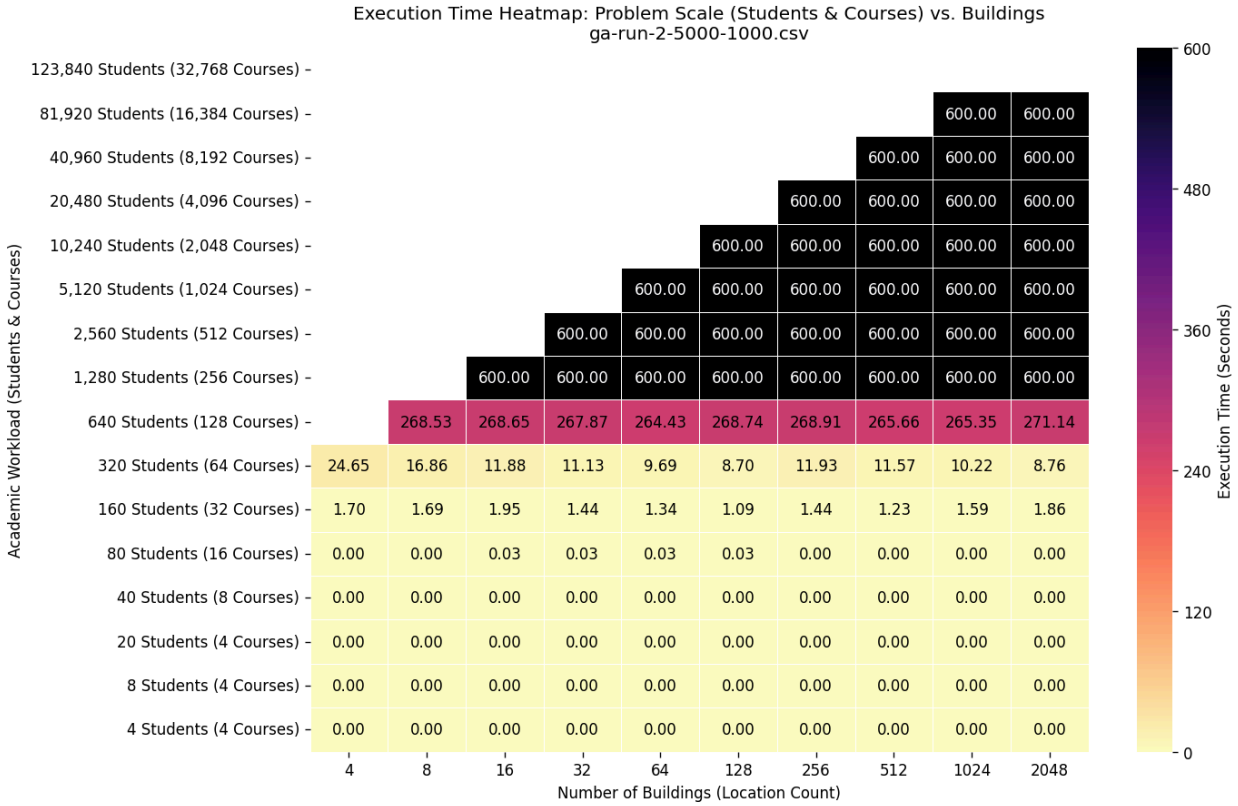
Heatmap 7: gc-run-3_heatmap (Graph Coloring run #3)



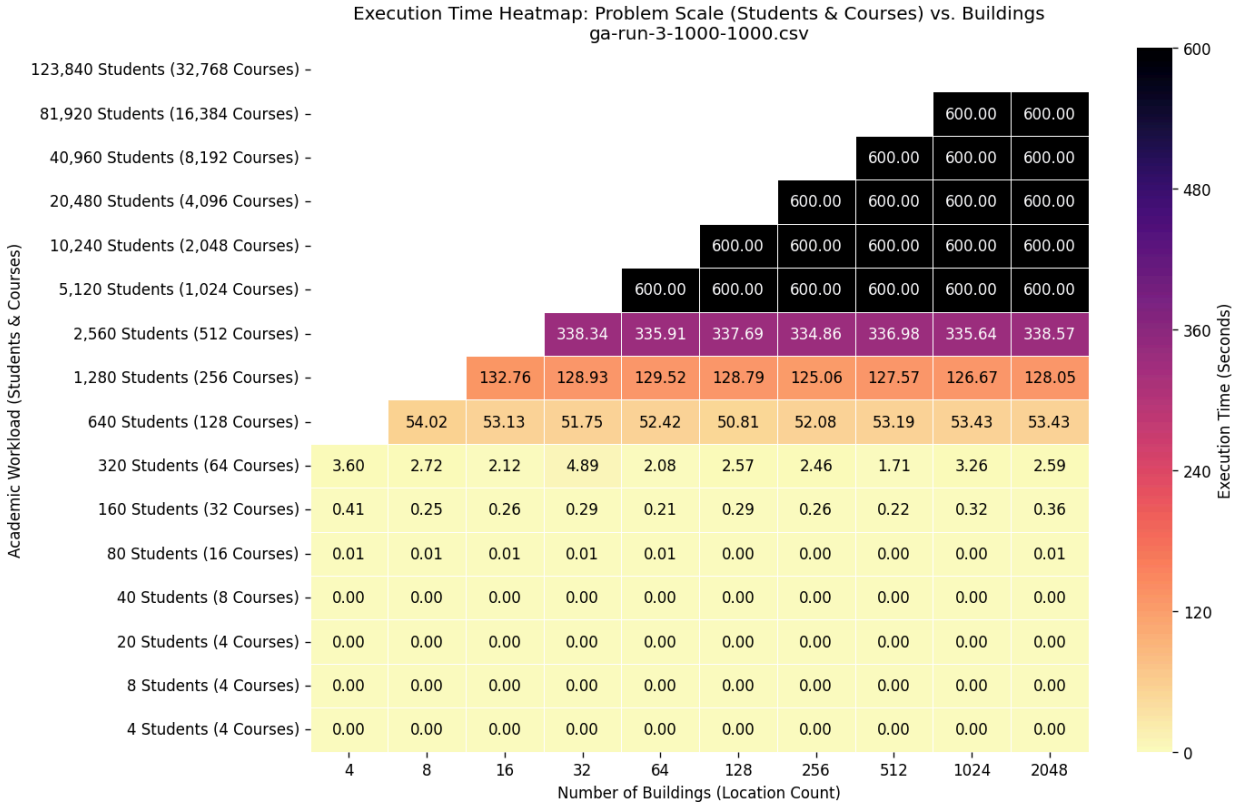
Heatmap 8: ga-run-1-5000-5000_heatmap (Genetic run #1, 5000 pop., 5000 gens.)



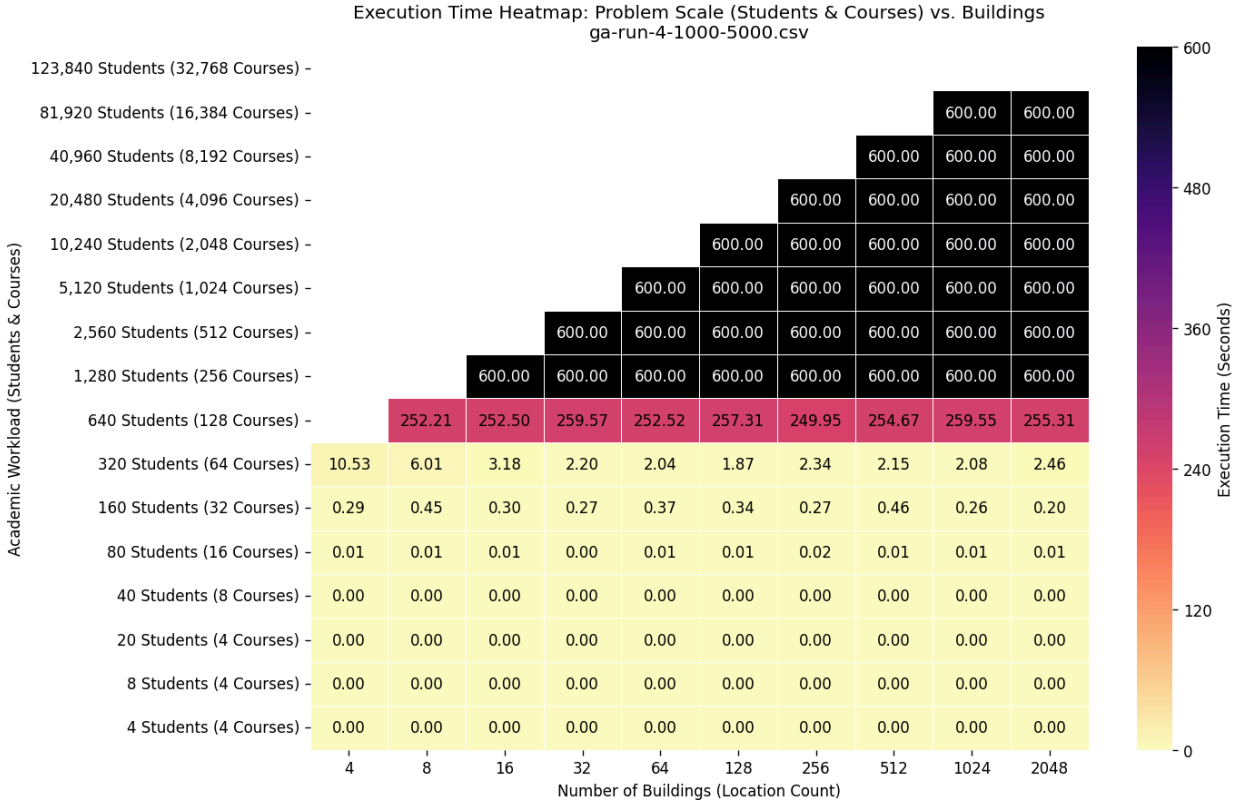
Heatmap 9: ga-run2-5000-1000_heatmap (Genetic run #2, 5000 pop., 1000 gens.)



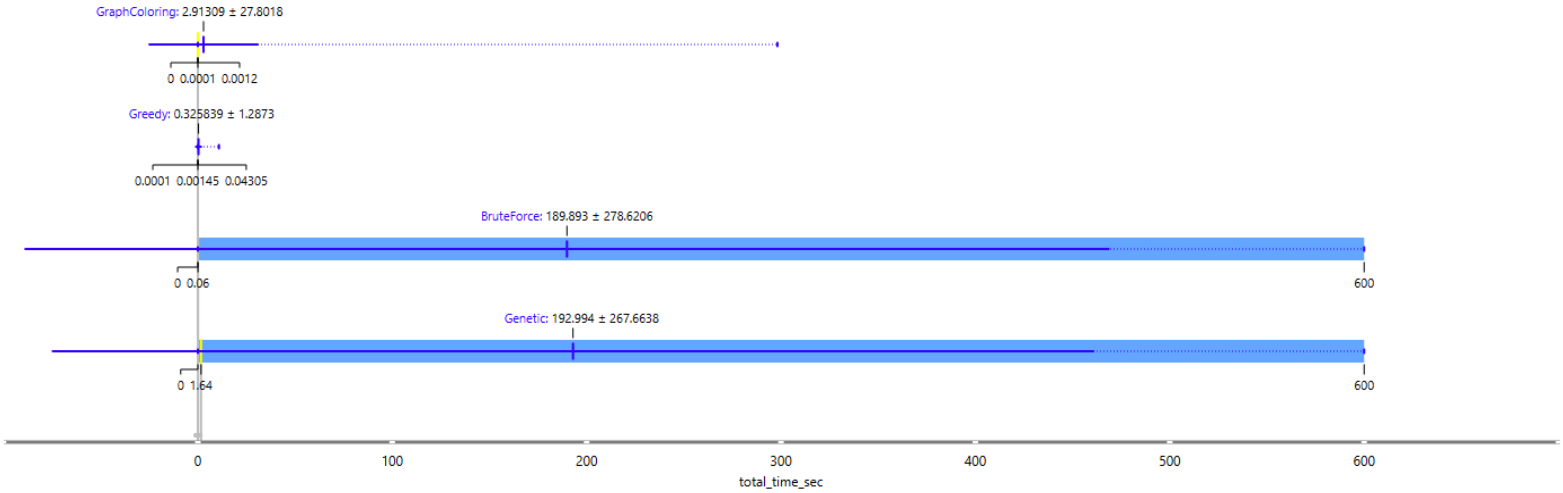
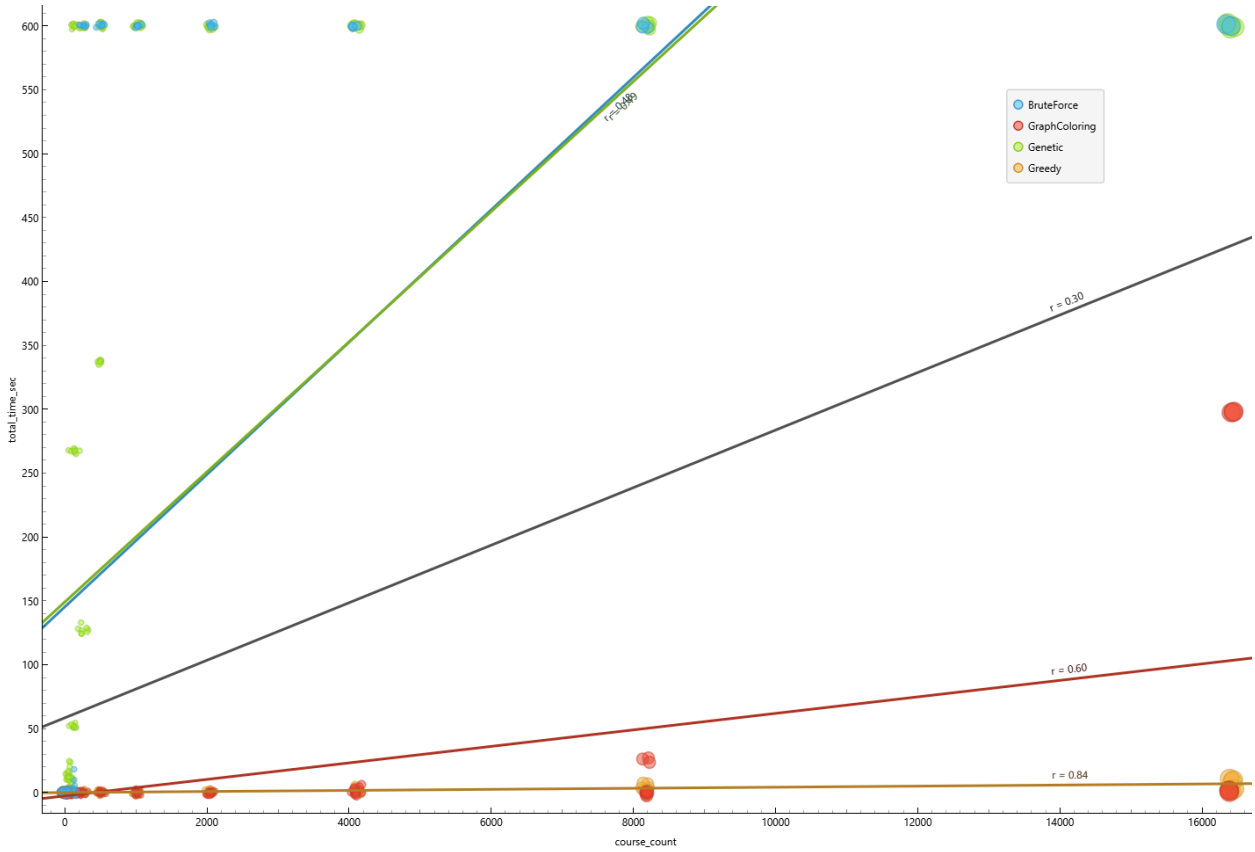
Heatmap 10: ga-run3-1000-1000_heatmap (Genetic run #3, 1000 pop., 1000 gens.)



Heatmap 11: ga-run4-1000-5000 (Genetic run #4, 1000 pop., 5000 gens.)



Boxplot 1: Time comparison of all algorithms (except ga-run-4-1000-5000)



Scatterplot 1: Exam count vs. Runtime for all algorithms (except ga-run-4-1000-5000)